SECTION 3 · SUBSECTION 3.2
SYMBOL EXTRACTION

3.2.1 INTRODUCTION.

Symbol extraction turns text into a language-neutral inventory. Each file is parsed by a tree-sitter grammar; three extractor families then walk the tree and emit functions, classes and imports into a *SymbolTable*. The one idea the reader must leave with is that later stages never see syntax trees. They see only the flat records defined in *src/parsers/base.py* (figure 3.2-1).

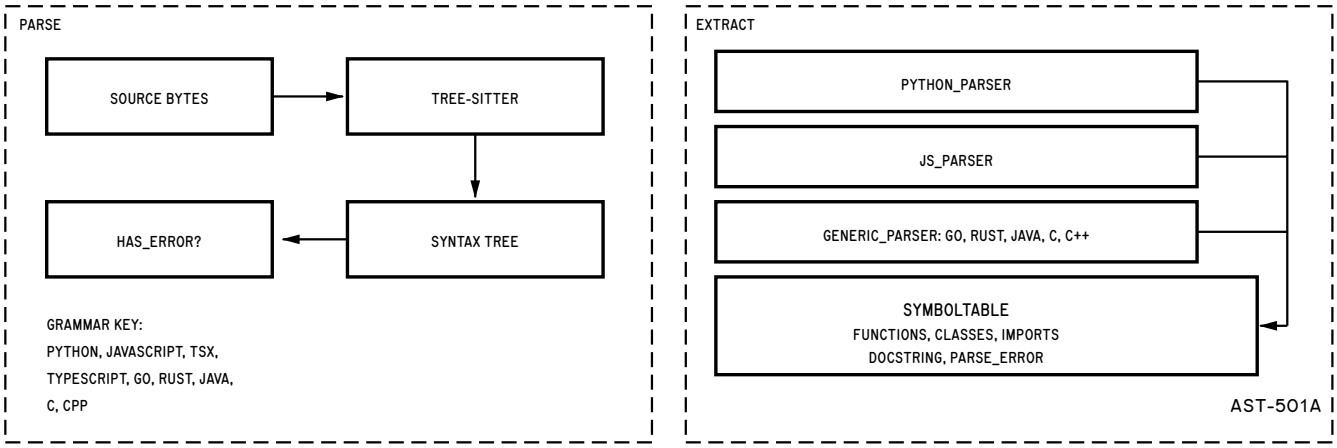


Figure 3.2-1. Parse and Extract Paths for Eight Languages

3.2.2 FUNCTIONAL DESCRIPTION.

run builds the set of manifest paths, selects files to parse, and submits them to a thread pool sized *min(12, CPU count, file count)*. Each worker calls *_parse_single_file*, which selects a grammar key, parses bytes, and dispatches to *extract_python*, *extract_js* or *extract_generic*. Python methods are recorded both as functions and as class method names; JavaScript arrow functions and exported declarations are recognised; the generic extractor handles receiver-based Go methods, Rust impl blocks, Java constructors and C declarators. After parsing, *_resolve_internal_imports* marks each import as internal or external against the manifest.

3.2.2.1 Error Tolerance.

A tree-sitter syntax error does not discard the file. The extractor still emits whatever symbols it can and sets *parse_error=True* with a detail string. An exception during extraction produces an empty table flagged as an error. Downstream, the file is skipped by retrieval and forced to CRITICAL risk, so a parse failure is visible rather than silent.

```
# src/agents/ast_parser.py, _parse_single_file (condensed)
grammar_key = "TSX" if path.endswith(".tsx") else lang
parser = get_parser(grammar_key)          # None -> skip, not error
tree = parser.parse(source_bytes)
has_error = tree.root_node.has_error      # partial tree still usable
docstring, functions, classes, imports = extract_python(tree, source_bytes)
st = SymbolTable(file_path=path, language=lang, module_docstring=docstring,
                 functions=functions, classes=classes, imports=imports,
                 parse_error=has_error)   # error is data, not an exception
```

3.2.3 PERFORMANCE AND DESIGN DATA.

Table 3.2-1. Grammars and Symbol Coverage

Language	Package	Coverage
Python	tree-sitter-python	functions, async, classes, methods, imports
JavaScript	tree-sitter-javascript	declarations, arrows, classes, imports
TypeScript	tree-sitter-typescript	TypeScript and TSX grammars
Go	tree-sitter-go	functions, methods, structs, imports
Rust	tree-sitter-rust	functions, impls, use declarations
Java	tree-sitter-java	classes, methods, imports
C / C++	tree-sitter-c / cpp	functions, structs, includes
Config / docs	none	skipped after ingestion

CAUTION

Symbol coverage is not semantic type resolution. A dynamic import, a generated macro, or a name assembled at run time will not appear. The absence of a symbol in table 3.2-1's output is not evidence that the symbol does not exist in the target repository.

PARSE CHECK	MAINTAINER
1 READ - symbol_tables count	
2 COMPARE - manifest count	
√ PARSE_ERROR - false for sample	
3 OPEN - one function record	
4 CONFIRM - line range matches file	

AST